Certificates and Cover Compression for Deterministic Matching

Yilin Liu

26 October 2026

Abstract

We give a deterministic reference implementation for maximum matching in general graphs, with independently verifiable optimality certificates and original input-edge identifiers. A cover-based reduction preserves every matchable cover subset and reduces each blossom solve to at most four times the component matching number. Including certificate construction, the running time is $O(n + m \log(m + 2) + \sum_i \nu(G_i)^3)$, with linear record storage. If a retained component is bipartite, both its crossing compression and final solve have matching-number exponent 5/2, and its certificate is constructed in linear additional work. The certificate verifier performs no matching search. This paper proves the cover compression and the simultaneous failed-forest certificate extraction, proves the nonrecursive bipartite phase algorithm and its matching-number phase bound, and states the boundary to the classical blossom algorithm used for the other components.

Scope and evidence. The executable solver uses the standard-library implementation linked from the repository's reference directory. Its classical augmenting-path method is Edmonds's *Paths, Trees, and Flowers*, Canadian Journal of Mathematics 17 (1965), Section 4, doi:10.4153/CJM-1965-045-4. The independent odd-component upper bound proves the optimality of each accepted certificate. Finite enumeration supports implementation review; it is distinct from the universal proofs below. This is the complete polynomial reference package, with its stated word-operation and identifier encoding model.

Part I

Independent optimality certificates

This note proves the certificate extraction implemented by `_failed_search_forest` and `_barrier` in matching.py. Given a maximum matching of size ν on a simple graph with k vertices and h edges, extraction takes $O(h + k(\nu + 1))$ additional work, at most $O(k^2)$. This preserves the reference solver's cubic bound on general components. The certificate is checked by the independent verifier, which performs no blossom search.

The public solver uses this extraction on nonbipartite retained kernels. On a bipartite kernel, a separate local proof constructs a minimum vertex cover in `O(k+h)` work and proves that it is a tight certificate in the same format. This avoids a cubic certificate cost in the faster bipartite branch. The theorem below continues to apply to `_failed_search_forest` and `_barrier` on any simple graph.

The construction is a classical alternating-forest method. The resulting sets give the Gallai–Edmonds decomposition, whose historical source is Gallai (1964), Theorem (E.1). The proof below establishes the required extraction and upper-bound equality directly; it does not assume that decomposition theorem.

I.1 Statement and algorithm

Fix a maximum matching M of the simple graph H. Let f be the number of vertices exposed by M, meaning unmatched by it. Start one Edmonds alternating forest with all exposed vertices as roots. Keep M fixed throughout the search.

Every root is initially **outer**. An edge from an outer vertex to an unreached matched vertex grows the forest by that **inner** vertex and its matched outer child. An edge between different outer blocks in the same tree contracts the usual odd blossom. Every original vertex in the new block becomes outer; newly outer vertices enter the scan queue. An edge between outer vertices in different trees would give an augmenting path, so the implementation rejects that state as inconsistent with a maximum input matching.

After exhausting the queue, let O be the set of all original outer vertices and I the remaining inner vertices. Return $A = I$.

Theorem 1. The algorithm leaves M unchanged and returns a set A satisfying

$$2|M| = |V(H)| - o(H - A) + |A|,$$

where $o(H - A)$ is the number of odd connected components after deleting A. It uses $O(h + k(\nu + 1)) = O(k^2)$ time and $O(k + |E(H)|)$ storage. Moreover,

$$O = \{v : \nu(H - v) = \nu(H)\}, \qquad I = N_H(O) \setminus O.$$

The implementation first tests $A = \varnothing$ and skips the forest search when that set already gives the displayed equality. This shortcut returns a valid barrier, although it need not be the particular set $N_H(O) \setminus O$. The theorem describes the forest search when it is used; the independent verifier accepts either tight barrier.

I.2 Invariants of the contracted forest

A **block** is either a singleton or the set of original vertices inside a current contracted blossom. The quotient graph has one vertex per block. At every stage the search maintains these invariants:

1. Each outer block is connected, has odd order, and is factor-critical: deleting any prescribed vertex leaves a perfect matching inside that block. Its current M-edges form a near-perfect internal matching whose only internally exposed vertex is the block's base.
2. **Only the base can have a matched edge leaving an outer block.** A nonroot outer block's base is matched to its inner parent. A root block has no external matched edge and contains exactly one original exposed vertex of M.
3. Every inner vertex is a singleton and has exactly one matched outer child. Thus every nonroot outer block has one inner parent, and every inner vertex has one such child. There is one more outer block than inner vertices in each quotient tree.
4. The original members of different blocks are disjoint. Every original member of an outer block is marked outer and has its adjacency scanned once after receiving that mark.

These statements initially hold for the singleton roots. Growing a tree adds one inner singleton and its matched outer child, preserving the matching structure and the difference between the two node counts.

For a contraction, take an edge between outer blocks in the same tree. The least common tree ancestor of its endpoints is outer: an inner vertex has only one child and cannot be the

branching ancestor. The two paths to this ancestor, together with the new edge, form an odd cycle in the quotient, with one more outer block than inner singleton. If the cycle contains a outer blocks, it contains $a - 1$ inner vertices. Replacing them by one outer block removes $a - 1$ of each type, preserving the count exactly.

The new block is factor-critical. More generally, consider an odd cycle of disjoint odd factor-critical blocks joined by selected connecting edges. After deleting a vertex in one block, match that punctured block internally. Pair the remaining even number of blocks around the cycle. For each pair use its connecting edge, and match both blocks internally after deleting the connecting endpoints. This is a perfect matching of the union minus the prescribed vertex. The argument applies recursively to nested blossoms and uses only actual graph edges.

The fixed matching also has the required base property after contraction. Each inner vertex on the alternating cycle is matched to its outer child on that cycle. Each constituent outer block is internally matched except at its base. Consequently all bases on the cycle are matched internally to the new block except the ancestor block's base. That base retains its previous external matched edge, if any. No second external matched edge appears.

For the array implementation, `base[v]` identifies the current quotient block. To climb from an outer block to its outer grandparent, first use the matched edge at its base and then the `parent` edge of that inner vertex. These are exactly the two steps used in `common_base`. `mark_branch` rewrites the alternating routes along the two cycle branches; the later scan identifies all their blocks with the ancestor base. The ancestor's external matched edge is unchanged. Tree branches attached to the contracted cycle remain attached to its new block. The root label `owner` remains unchanged within that tree. This maintains the quotient forest and the unique external matched-edge invariant through every parent rewrite.

I.3 Lifting paths through the blocks

Every vertex of an outer block can be exposed by a matching of the same cardinality as M, preserving the other roots. To see this, follow the quotient tree path from its root block to the target block. Remove the matched edge from each inner vertex to the next block base, and add the preceding unmatched tree edge into that inner vertex.

In each nonfinal block on this path, the new crossing edge uses one known attachment vertex x. Replace the internal matching by a perfect matching of that block minus x, using factor-criticality. In the final block leave the prescribed target vertex exposed. Other blocks and off-path matching edges stay as they were. Exactly one original root has been matched and the target has been exposed; cardinality is unchanged.

The symmetric difference with M contains a simple even alternating path from that root to the target. Its first edge is unmatched by M and its last edge is matched by M; a zero-length path is allowed. This is the required lifting argument. It does not assume that an arbitrary projected walk through nested blossoms is already simple.

If an edge joined outer vertices in different trees, the two lifted paths would be vertex-disjoint because their trees have disjoint blocks. Joining them by that edge gives an augmenting path between two exposed roots. Such a path would increase $|M|$, contradicting maximal cardinality. This proves that the explicit cross-tree rejection cannot occur for a correct maximum matching.

I.4 Exhaustion gives a tight odd-component bound

Let b be the number of final outer blocks. Summing the quotient-tree count over all f roots gives

$$b = |I| + f.$$

There is no edge between distinct final outer blocks. Such an edge would have been scanned when its later-labelled endpoint became outer. It would contract blocks within a tree or give an augmenting path between trees. Later contractions only merge blocks.

Also $N_H(O) \setminus O = I$. An unreached neighbor would grow a tree, while every inner vertex has an outer parent. Let $U = V(H) \setminus (O \cup I)$. Every vertex in U is matched by M to another vertex of U: all exposed vertices were roots, and every reached matched pair lies inside an outer block or is an inner/outer parent-child pair. Hence M restricted to U is a perfect matching.

After deleting I, the components are the b connected odd outer blocks and the components of $H[U]$. Each latter component has a perfect matching and therefore even order. It follows that

$$o(H - I) = b = |I| + f.$$

Since $2|M| = k - f$, this proves the equality in Theorem 1.

For completeness, the equality is a self-contained optimality certificate. For any proposed set A, each odd component of $H - A$ needs either an unmatched vertex or a matching edge to A. At most $|A|$ components can use such matching edges. Therefore every matching has size at most $(k - o(H - A) + |A|)/2$. A feasible matching attaining that bound is maximum. The independent verifier uses only this argument and the separately proved exact kernel conditions.

I.5 Why the outer set is exactly the exposable set

Section 3 shows that every vertex of O can be exposed by a maximum matching. Conversely, any matching leaves at least $b - |I| = f$ vertices of O unmatched: each odd outer block needs an unmatched vertex or a distinct matching endpoint in I. A maximum matching has exactly f exposed vertices in the whole graph, so it cannot expose a vertex outside O. Thus O is exactly the set of vertices exposable by a maximum matching.

A vertex is exposable by a maximum matching precisely when $\nu(H-v) = \nu(H)$. Equivalently it is the endpoint of a simple even M-alternating path from an exposed root. One implication follows by flipping that path; the other follows by taking the symmetric difference of M with a maximum matching that exposes the vertex. Both matchings are maximum, so their path components cannot augment either matching. This proves the remaining claims of Theorem 1 without invoking an external decomposition theorem.

I.6 Cost and executable evidence

Every vertex becomes outer at most once and has its adjacency scanned once. A matched edge can create at most one inner singleton during the whole search. When its first endpoint becomes inner, the other endpoint becomes outer, and outer status is permanent. If that inner endpoint is later absorbed by a blossom, both endpoints remain outer; the edge cannot create a second inner vertex. Thus there are at most $\nu = |M|$ inner-vertex births.

Every nontrivial contraction has at least one inner singleton on its odd quotient cycle. All such vertices become outer permanently. Charge each contraction to one of these absorbed inner vertices. No vertex can be charged again, so there are at most ν contractions. Ancestor walks, branch rewrites and the full-array contraction scan each cost $O(k)$. Initialization and ordinary adjacency scans cost $O(k + h)$. The total is $O(h + k(\nu + 1))$, with linear additional storage. Constructing I and checking its odd-component count also takes $O(k + h)$ work.

For the two-stage reference solver, the first edge kernel has $h, k = O(c^2)$ and $\nu \le c$. Extraction on that original kernel therefore costs $O(c^3)$, even when the blossom solve took place on its smaller $2c$-vertex induced graph.

The dedicated search checker compares the actual production function against the deletion-defined exposable set using independent subset dynamic programming. It checks all simple graphs through six vertices and 500 seeded graphs on seven through thirteen vertices. It also verifies the terminal count, the factor-criticality of each outer block, its unique external matched edge, the perfect matching on U, and the exact odd-component count. For graphs through nine vertices it independently enumerates simple even alternating paths. Rejection of a nonmaximum input matching uses explicit production exceptions and remains active under optimized Python. The compression checker additionally checks the ν contraction bound on the production search after lifting the smaller-graph matching back to the original kernel.

The full reference test suite additionally checks every public solver result with the independent certificate verifier, including native loops, parallel records, isolated vertices, invalid certificates and CLI execution. These are finite implementation checks supporting the proof and review; they are not proof-assistant formalization.

Part II

Cover compression and matching-number complexity

The reference solver first builds the proved c-neighbor edge kernel, then reduces the number of vertices further using a maximum matching in its bipartite crossing graph. The resulting default time bound, including an optional optimality certificate, is

$$O\left(n + m\log(m+2) + \sum_i \nu(G_i)^3\right),$$

where G_i ranges over nonempty normalized input components. No matching-number estimate is supplied as input. This is an ordinary polynomial implementation improvement. The structural reduction below restates the earlier checkpoint proof described next. The separate faster research composition uses its own cycle-search construction and does not charge this implementation's polynomial crossing solve to its fast bound.

The bipartite refinement improves a component's cubic solve term to `O(nu(G_i)^(5/2))` whenever its first retained kernel is bipartite, including certificate generation. The crossing matching itself now uses Hopcroft–Karp; Section 2 accounts for this required improvement.

Historical source. Lemma 1's right-endpoint exchange and the resulting $2c \le 4\nu$ vertex bound were already proved in A linear-vertex refinement of the matching-size kernel (linear_vertex_kernel_refinement.md), Sections 1–3, in the direct checkpoint of 24 October 2026. The fixed-epsilon and uniform checkpoints retain the identical proof. The present work supplies its standard-library implementation, the complete componentwise implementation bound, independent finite checks and certificate compatibility. The bipartite refinement adds the nonrecursive Hopcroft–Karp implementation and its matching-number analysis. See the checkpoint source map.

II.1 A small set of outside endpoints preserves all cover subsets

Lemma 1. Let C be any vertex cover of a simple graph H, put $I = V(H) \setminus C$, and let Q be a maximum matching in the bipartite graph containing the C–I edges of H. Let $T \subseteq I$ be the right endpoints matched by Q. Every subset of C matchable into I is matchable into T. Consequently the induced graph $J = H[C \cup T]$ satisfies

$$\nu(J) = \nu(H), \qquad |V(J)| \le 2|C|.$$

Proof. Fix a crossing matching P whose matched left set is a prescribed $S \subseteq C$. If P uses a right vertex $x \notin T$, then x is unmatched by Q. The component of $P \triangle Q$ starting at x is an alternating path beginning with a P-edge.

This path cannot end at a left vertex unmatched by Q: its first and last edges would both belong to P, giving an augmenting path for the maximum matching Q. It therefore ends at a right vertex $y \in T$ unmatched by P, with a final Q-edge. Flip P along the path. The matched left set remains exactly S; x becomes unmatched and y becomes matched. Every internal right vertex is incident to a Q-edge and already lies in T. Thus the number of P-used vertices outside T decreases by one. Repetition gives a matching of exactly S into T.

Now take any matching in the general graph H. Keep its C–C edges. Its crossing edges match a subset S of the remaining cover vertices; replace them using the preceding argument, preserving S. The union is a matching of the same cardinality in J. The reverse inequality follows from $J \subseteq H$. Finally $|T| = |Q| \le |C|$. □

The lemma holds for **any maximum** crossing matching. It preserves every matchable left subset simultaneously, including subsets chosen after T is fixed. A merely maximal crossing matching is insufficient.

II.2 Apply the edge kernel before computing the crossing matching

For a normalized component G_i, let C_i be the endpoints of a greedy maximal matching and put $c_i = |C_i|$. Then $\nu(G_i) \le c_i \le 2\nu(G_i)$. The first-stage c_i-neighbor kernel H_i has

$$h_i = |E(H_i)| \le \min\left\{ m_i, \binom{c_i}{2} + c_i^2 \right\}, \qquad k_i = |V(H_i)| \le c_i + c_i^2, \qquad \nu(H_i) = \nu(G_i).$$

Compute a maximum crossing matching in $H_i[C_i, V(H_i) \setminus C_i]$ and apply Lemma 1. The graph J_i given to the final matching search has at most $2c_i \le 4\nu(G_i)$ vertices. All its edges retain original native input indices and identifiers.

The crossing matching uses the implemented Hopcroft–Karp phase algorithm. Each phase freezes its starting matching, finds a maximal family of vertex-disjoint shortest augmenting paths, and flips those paths. The complete phase proof establishes correctness and the bound `O((k+h) sqrt(nu_cross+1))`, including one final failed BFS. Here `nu_cross <= c_i`, even though the crossing graph may have quadratically many vertices. Consequently `_compress_cover` costs `O(c_i^(5/2))` time and `O(c_i^2)` storage.

If the first kernel `H_i` is bipartite, its coloring restricts to `J_i`, and a second Hopcroft–Karp solve also takes `O(c_i^(5/2))`. Otherwise the subsequent blossom computation takes $O(c_i^3)$. Computing the crossing matching only **after** the edge kernel keeps all superlinear matching work on `O(c_i^2)` records; the original component is still scanned only linearly after normalization.

II.3 Certify on the original edge kernel

The public certificate format is unchanged. It lists the first-stage kernel H_i, its cover, the original chosen edge records, and an odd-component barrier in H_i. It does not ask the verifier to trust or execute the additional compression.

Lift the maximum matching of J_i back to H_i, marking discarded outside vertices unmatched. Lemma 1 proves that this is maximum in H_i. On a bipartite kernel, the alternating-reachability vertex cover gives a tight odd-component certificate in `O(k_i+h_i)` work. On a nonbipartite kernel, run the simultaneous failed-forest extraction there. Its sharper accounting is useful here.

Lemma 2. Given a maximum matching of size ν in a graph with k vertices and h edges, that extraction takes $O(h + k(\nu + 1))$ time and linear storage.

Proof. Each matched edge can create at most one inner singleton. When an endpoint first becomes inner, its mate becomes outer. Outer status is permanent. If the inner endpoint is later absorbed by a blossom, both endpoints remain outer; the same matched edge cannot create another inner vertex. There are therefore at most ν inner-vertex births.

Every nontrivial odd-cycle contraction absorbs at least one current inner singleton, permanently making it outer. Hence there are at most ν contractions. Each contraction's ancestor walks, temporary arrays and full-array scan cost $O(k)$. Ordinary adjacency scans cost $O(h)$ in total, and initial arrays cost $O(k)$. This gives the claimed bound. □

Here $k_i = O(c_i^2)$, $h_i = O(c_i^2)$, and $\nu(H_i) \le c_i$, so general certificate extraction costs $O(c_i^3)$; bipartite extraction costs `O(c_i^2)`. Its independently checked equality proves optimality on H_i and, by the verified edge-kernel conditions, on the original input. A faulty compression cannot make the unchanged verifier accept a suboptimal matching.

II.4 Complete time, storage and scope

Comparison-sort normalization costs $O(n + m\log(m+2))$. Constructing all first-stage kernels and handling components costs $O(n+m)$ after normalization. Both normal and certificate modes therefore take

$$O\left(n + m\log(m+2) + \sum_i c_i^3\right) = O\left(n + m\log(m+2) + \sum_i \nu(G_i)^3\right).$$

Working storage and certificate storage remain $O(n+m)$ records. Long edge-ID labels incur their actual comparison and encoding costs. The verifier still takes $O(n + m\log(m+2))$ word operations, with no matching search. Empty inputs and isolated vertices retain their previous handling.

The cubic sum is a uniform upper bound. More precisely, replace the term for every bipartite first kernel by `O(nu(G_i)^(5/2))`; both its compression and solve have that bound, and its certificate costs less. The bipartite proof states the resulting mixed-component formula.

The `--no-kernel` option disables both stages and solves the full normalized components. A bipartite component costs `O((n_i+m_i) sqrt(nu(G_i)+1))`; a nonbipartite component costs `O(n_i^3)`. Here `n_i,m_i` are the component's vertex and edge counts. The initial `O(n+m log(m+2))` term remains charged.

This compression is for the current static graph. It is not a replacement for the separately proved simultaneous mixed-failure kernel. A right endpoint set chosen before failures need not preserve later answers: with a one-vertex cover of a star, deleting the one retained crossing edge destroys its matching while another original leaf remains usable. The fault summary must remain intact until a particular failure instance has been fixed.

II.5 Evidence and a concrete size reduction

The production-helper checker tests all 33,868 simple graphs through six vertices in both fixed edge orders, followed by 500 seeded graphs on seven through thirteen vertices. It checks 68,236 covers, 70,578 possible maximum-matching right endpoint sets, and 2,882,175 left-subset ranks using independent subset dynamic programming. It invokes the actual compression code and verifies the matched-edge bound on contractions in the actual certificate search.

For a visible larger example, take a path on 20 cover vertices, list its 10 disjoint pair edges first, and attach 20 private leaves to each cover vertex. The first kernel has all 420 vertices and 419 edges. The new stage reduces the final bipartite solve to 40 vertices, returns 20 original edges, and certifies the answer on the original 420-vertex kernel. This case is included in the full solver checker.

The finite checks support implementation review and the displayed local proofs. They do not certify the separate fast research hierarchy.

Part III
Bipartite acceleration and its certificate

This proves the bipartite branch of the ordinary, unconditional matching reference implementation. It does not implement the manuscript's faster deterministic cycle-search construction. The existing edge-kernel and right-endpoint compression proofs continue to supply their stated matching preservation properties.

III.1 Graphs and the exact bound

Let `n` count the named vertices and `m` the native input edge records. Normalize the graph by removing loops and retaining the earliest input record of every unordered pair, while preserving its original ID for output. Let the nonempty connected components of this graph be `G_i`, and write `nu_i` for their maximum matching sizes. In each component, the endpoints of a greedy maximal matching form a cover `C_i` of size `c_i <= 2 nu_i`.

The first compression retains all cover–cover edges and the first `c_i` distinct outside neighbors of each cover vertex. Its graph `H_i` has `O(c_i^2)` vertices and edges, no more edges than the original component, and the same maximum matching size. The second compression takes a maximum matching `Q_i` in the bipartite crossing graph `C_i` versus `V(H_i) - C_i`. If `T_i` comprises its matched right endpoints, the induced graph `J_i = H_i[C_i union T_i]` has at most `2 c_i <= 4 nu_i` vertices and preserves every cover subset that could be matched to the outside. Hence it preserves the general matching number as well.

The implementation uses the bipartite phase algorithm of John E. Hopcroft and Richard M. Karp, *An n^(5/2) Algorithm for Maximum Matchings in Bipartite Graphs*, SIAM Journal on Computing 2(4), 225–231 (1973). The published paper gives the classical `O((V+E) sqrt(V))` bound. Sections 2–4 below give the specific implementation invariants and the matching-number refinement needed here; the refinement is not obtained by substituting `nu` for the vertex count without proof.

Hopcroft–Karp computes `Q_i`, and a breadth-first two-color test on `H_i` decides the final branch. If `H_i` is bipartite, its coloring restricts to `J_i`, which is solved by Hopcroft–Karp. Otherwise the existing Edmonds blossom implementation solves `J_i`. Both return the original

input IDs after lifting through the unchanged index maps. Testing `H_i` rather than only `J_i` also permits the linear certificate algorithm in Section 5.

Let `B` be the set of components whose retained `H_i` is bipartite. The total operation count, with or without optional optimality certificates, is

$$O\left(n + m\log(m+2) + \sum_{i\in B}\nu_i^{5/2} + \sum_{i\notin B}\nu_i^{3}\right).$$

Every originally bipartite component belongs to `B`; occasionally deletion of surplus edges makes `H_i` bipartite even when `G_i` was not. This broader classification is safe because both kernel preservation and certification refer to `H_i`. The bound is a componentwise operation bound on the stated integer-index model; long ID comparisons and input/output representation bytes retain the charges in the reference README.

III.2 The phase implementation

Consider a bipartite graph with fixed left and right sides, `k` vertices, `h` edges, and a current matching `M`. Only left adjacency lists are needed. An augmenting path begins at an unmatched left vertex, ends at an unmatched right vertex, and alternates between unused and matching edges. If it has `d` matching edges, it has `2d+1` graph edges. This `d`, not the total path length, is the unit used in the implementation.

The BFS initializes every free left vertex at level zero. From a left vertex `u` at level `d`, an edge outside `M` either reaches a free right endpoint, producing a candidate terminal level `d`, or reaches a matched right vertex whose partner is put at level `d+1`. The first terminal level is the minimum number of matching edges on any augmenting path. Vertices at greater levels need not be scanned. If there is no terminal, the algorithm returns `M`.

Otherwise, `phase_mate` freezes the matching for the entire phase. An explicit-stack DFS follows only this fixed layered graph: a matched right vertex advances to its partner at the next left level; a free right vertex is accepted only at the minimum terminal level. Each left adjacency list has a cursor advanced whenever an edge is examined. A left vertex exhausted without a terminal is permanently marked unavailable for this phase. Every vertex on a successful path is also marked unavailable.

Exhausted vertices may safely remain unavailable: subsequent successful paths only delete vertices from this fixed directed acyclic layered graph. They never introduce a new path from a previously exhausted vertex. The strictly increasing left levels also prohibit a repeated vertex within one DFS path. Successful paths are therefore vertex-disjoint. The implementation may flip them immediately because every subsequent search reads `phase_mate`, and the unavailable marks prevent intersecting a flipped path.

Each originally free left root is searched unless already used. If, at the end, a shortest augmenting path disjoint from all selected paths remained, its free root would have been exhausted even though it had a route to a terminal in the remaining graph. This contradicts the DFS rule. The selected paths are consequently a maximal vertex-disjoint family of shortest augmenting paths. A reachable phase must have at least one such path; the production code raises an explicit error if this invariant fails.

The BFS, initialization, matching snapshot, cursors, unavailable marks, and path flips cost `O(k+h)` per phase. A successful path's vertices are visited only once during its flip; a failed left vertex never reenters a later DFS. Every left adjacency cursor advances at most its degree. The implementation uses no recursion, so the bound does not require a bounded augmenting-path depth.

III.3 Why the next shortest path is longer

For completeness, use the standard unit-capacity matching network: arcs `s -> L`, `L -> R` for graph edges, and `R -> t`, each with capacity one. The integral unit flow of `M` has an `s`–`t` residual path exactly when the graph has an augmenting path. Such a graph path with `d` matching edges becomes a residual path of `2d+3` arcs. Simplicity removes any irrelevant return to `s` or visit to `t` before the final endpoint.

Before one phase, let `ell(x)` be residual distance from `s`. Every residual arc from a reachable vertex satisfies `ell(v) <= ell(u)+1`. A shortest `s`–`t` path increases this distance by exactly one at each step. Augmenting the chosen disjoint shortest paths introduces only reversals of such arcs; every newly introduced arc goes backward by one old level. Every unchanged arc still advances by at most one. No newly reachable vertex can have had infinite old distance: unchanged arcs out of the old reachable set could already be followed, and new arcs have endpoints on the chosen paths.

It follows that a new augmenting residual path is at least as long as the old shortest one. If its length were equal, every arc would advance by one old level. It could not use any reversed path arc. Moreover it could not pass through a vertex of a chosen path using only unchanged forward arcs: each left vertex on that path had exactly one incoming residual arc (its old matching edge, or its source arc when free), and that arc was used and reversed; equivalently, each right vertex had exactly one outgoing residual arc (its old matching edge, or its sink arc when free), also used and reversed. Thus an equally short new path would be an old shortest path vertex-disjoint from the entire chosen family. Section 2's maximality rules this out.

The minimum `d` therefore strictly increases after every successful phase that leaves another augmenting path. Its integer value increases by at least one. This is the progress fact used by the phase count, not an assumption that any single arbitrary augmentation increases the shortest path length.

III.4 Matching number controls the phase count

An empty graph is handled in constant time. For a nonempty vertex array, let `nu` be the unknown optimum matching size. For `nu=0`, the initial BFS already fails and costs `O(k+h)`. Suppose `nu>=1`, and set `r = ceil(sqrt(nu))`. After `r` successful phases, if the algorithm has not terminated, every augmenting path contains at least `r` matching edges by Section 3 and the initial lower bound zero.

Compare the current `M` with any optimum `M*`. Their symmetric difference is a vertex-disjoint union of alternating paths and cycles. No component can have one more `M` edge than `M*` edges: flipping that component would increase the maximum matching `M*`. Components with one more `M*` edge are augmenting paths for `M`, and there are exactly `nu - |M|` of them. Every such path uses at least `r` distinct edges of `M`; the paths are edge-disjoint. Consequently

$$r(\nu - |M|) \le |M| \le \nu.$$

At most `floor(nu/r)` further successful phases are possible, since each increases matching size by at least one. There are at most `r + floor(nu/r) <= 2 ceil(sqrt(nu))` successful phases in total. **One final failed BFS is also charged**, including the case where the last successful phase produces a maximum matching. This gives

$$O((k+h)\sqrt{\nu+1})$$

operations and O(k+h) storage. This also proves optimality: on termination there is no augmenting path, whereas the symmetric-difference argument would supply one if a larger matching existed.

For the crossing graph used by compression, nu_cross <= c, even though its number of vertices may be Theta(c^2). Thus the crossing computation costs O(c^(5/2)), rather than the O(c^3) cost of sequentially augmenting once for every cover vertex. The induced J has O(c) vertices and O(c^2) edges, and its matching number is nu <= c, so its bipartite solve also costs O(c^(5/2)). Accelerating only the second solve would not suffice: the old compression step would otherwise keep the total cost cubic.

For --no-kernel, a bipartite full component is solved in O((n_i+m_i) sqrt(nu_i+1)); nonbipartite full components retain their ordinary O(n_i^3) blossom bound. Normalization and explicit vertex/output scans remain charged separately.

III.5 A linear certificate on the first kernel

Lift the resulting maximum matching from J to the retained graph H. Its matching number is unchanged by compression. Suppose the two-color test gave sides L,R for H. Start an alternating reachability search at every unmatched left vertex. Follow nonmatching edges from left to right, and the unique matching edge from a reached matched right vertex back to the left. Let Z be the reached vertex set. Reaching an unmatched right vertex would exhibit an augmenting path and is an explicit production error.

Set A = (L - Z) union (R intersect Z). This is a vertex cover. For an edge lr, if l is not reached then l is in A. If l is reached and lr is not a matching edge, the search reaches r, so r is in A. If it is a matching edge, the matched left vertex l could only have been reached through its partner r, giving the same conclusion; a root is unmatched.

For each matching edge, its endpoints are either both reached or both unreached. Hence exactly one endpoint belongs to A. Every unmatched left vertex is reached and is not in A; every reached right vertex is matched, so there is no unmatched right vertex in A. Therefore |A| = |M| = nu. This proves minimum cardinality directly, because any vertex cover must meet all nu disjoint matching edges.

The existing certificate format records a vertex set whose odd-component bound is tight. A vertex cover A is exactly such a set: H-A is independent, so all k-|A| remaining vertices are singleton odd components. Consequently

$$2|M| = k - \mathrm{odd}(H - A) + |A| = 2|A|.$$

The separate verifier already checks the matching, kernel preservation, and this odd-component equality. It needs no new field, no bipartite algorithm, and no change of certificate format. The producer additionally checks the lifted matching's validity, the edge-cover condition, and |A|=|M| by explicit rejection conditions; none depends on assert. Coloring, alternating reachability, these checks, and output all cost O(k+h)=O(c^2).

Nonbipartite H continues to use the previously proved general forest barrier extraction in O(h+k(nu+1)), which is O(c^3) on the first kernel. The general case's bound is unchanged. The bipartite branch needs neither that cubic certificate computation nor an external decomposition theorem.

III.6 Finite evidence and scope

The bipartite checker compares the actual phase solver with an independent subset-enumeration matching oracle on every bipartite graph with at most four vertices per side. It checks matching validity, the phase-count bound, the final failed BFS count, and the produced minimum cover. Seeded cases reverse the side names and adjacency order and exercise both public kernel modes with the unchanged independent optimality verifier.

A separate construction forces a second-phase augmenting path containing 2,400 matching edges, beyond Python's usual recursion limit. Native cases include ignored loops, parallel and reversed records, isolated vertices, and distinct integer versus string IDs. Another fixture has a nonbipartite original component but bipartite retained `H`, checking the branch's exact scope. Invalid side masks, invalid matchings, and a valid nonmaximum matching are rejected explicitly. The existing exhaustive general-graph, compression, certificate-forest, and CLI suites continue to exercise the blossom fallback.

These are finite implementation checks supporting review of the local proofs. They do not certify all executions or establish the manuscript's complete faster cycle-search construction or its asymptotic work bound.

Attribution and version. Yilin Liu and GPT-6 Astra Pro. *Algorithm Frontier*, 26 October 2026. The source files, executable checks and their exact hashes are recorded by the repository's paper build manifest. External peer review and proof-assistant formalization are not claimed.